

NEXAURA

NexClinic

Integrated Healthcare Management Platform

DEVELOPER GUIDE

Architecture, Setup, Integration, Testing, Deployment, and Maintenance

Document Type	Developer Guide
Version	1.0
Prepared Date	24 September 2026
Intended Audience	Developers, Maintainers, Testers, Reviewers, Deployment Operators
Project	CO2060 – Semester 3 Project

PROJECT TEAM

M.T. Dineth
H.M.U.A. Perera
S.M.D.S.B. Samarakoon
J.G.G. Methmaka

Table of Contents

I. System Overview

- Business Purpose
- Major User Roles
- Major Functional Areas
- High Level Architecture
- Main Data Flow

II. Architecture

- Frontend
- Backend
- Database
- Real Time Services
- Background Services
- Email Service
- File Storage
- Component Architecture Diagram
- Request Flow
- WebSocket Flow
- Notification Flow

III. Technology Stack

IV. Suggested Project Organization

V. Local Development Setup

- Required Software
- Backend Setup
- Frontend Setup
- Database Configuration
- Redis Setup
- Email Development Setup
- Migrations
- Creating an Administrator Account
- Starting Local Services

VI. Environment Variables

VII. Authentication Architecture

- Registration and Registration OTP
- Login and Login OTP
- JWT Access and Refresh Tokens
- HTTP Only Cookies and Token Refresh
- Logout
- Password Reset
- Role Validation and Authorization
- Frontend Route Guards vs. Backend Permissions
- Object Level Authorization

- WebSocket Authentication
- OTP Expiry and Attempt Controls
- Authentication Sequence Diagrams

VIII. Domain Model

- Conceptual Entity Relationship Diagram

IX. REST API Design

X. Appointment and Scheduling Logic

- Slot Creation: Direct and Template Generated
- Date and Time Validation
- Hospital and Doctor Relationships
- Capacity
- Booking Transactions and Duplicate-Booking Prevention
- Appointment States
- Clinical Record Creation
- Auditability
- Time Zone Testing
- Concurrent Booking Risks

XI. Real Time Chat

- WebSocket Connections and Authentication
- Eligible Chat Threads and Participants
- Message Delivery and Attachments
- Read State
- Thread Closing
- Error Handling and Reconnection
- Allowed Origins, Hosts and Production WSS

XII. Notifications and Background Tasks

- Notification History and Read State
- Real Time and Email Delivery
- Announcements, Target Audiences and Preferences
- Celery Tasks, Redis and Worker Processes
- Periodic Tasks and Retry Behavior
- Operational Logging
- Notification Flow Diagram

XIII. File Uploads

- File Type and File Size Validation
- Authorization
- Secure File Names and Malware Considerations
- Persistent Storage and Signed URLs
- Privacy, Retention and Deletion

XIV. Frontend Development

- App Router, Public and Protected Routes
- Role Specific Workspaces and Shared Components
- API Route Handlers and the Backend for Frontend Pattern

- Secure Cookie Forwarding
- TypeScript Response Models
- Forms, Validation and Interface States
- Responsive Layouts and Accessibility

XV. Testing and Verification

XVI. Deployment

- Production Environment Variables
- Migrations, Static Files and Persistent Media
- HTTPS and WSS
- Allowed Hosts, CORS and CSRF
- ASGI Deployment and WSGI Limitations
- Redis, Celery Workers and Scheduler
- SMTP
- Logging and Health Checks
- Backups and Rollback Planning
- Deployment Architecture Diagram

XVII. Security and Privacy

XVIII. Accessibility and Usability

XIX. Maintenance and Troubleshooting

XX. Change Checklist

XXI. Known Limitations

XXII. Future Improvements

XXIII. Glossary

XXIV. References

Screenshot Reference

List of Figures

- Fig. 1. High level request path from browser to backend services.
- Fig. 2. System component architecture (black and white schematic).
- Fig. 2. System component architecture (black and white schematic).
- Fig. 4. Login and token creation sequence.
- Fig. 5. Token refresh sequence.
- Fig. 6. WebSocket authentication sequence.
- Fig. 7. Conceptual entity relationship overview (simplified, black and white).
- Fig. 8. Notification generation and delivery flow.
- Fig. 9. Deployment architecture overview (black and white schematic).
- Fig. 10. Registration interface.
- Fig. 11. Login interface.
- Fig. 12. Patient dashboard.
- Fig. 13. Doctor dashboard.
- Fig. 14. Hospital administrator dashboard.
- Fig. 15. Doctor appointment workspace.
- Fig. 16. Advice chat workspace.
- Fig. 17. Notification centre.
- Fig. 18. API request example.
- Fig. 19. API response example.
- Fig. 20. Database migration output.
- Fig. 21. Backend test output.
- Fig. 22. Frontend lint output.
- Fig. 23. Frontend build output.
- Fig. 24. WebSocket connection verification.
- Fig. 25. Deployment health check.
- Fig. 26. Application logs.

List of Tables

Table 1. Document control information.

Table 2. Revision history.

Table 3. Approval and review record.

Table 4. Technology stack by architectural layer.

Table 5. Suggested top-level project organization.

Table 6. Environment variables, purpose, example values, and security considerations.

Table 7. Conceptual domain entities and their relationships.

Table 8. Representative REST API endpoint groups.

Table 9. Appointment state transitions and responsible actors.

Table 10. Verification matrix by test level.

Table 11. Maintenance and troubleshooting reference.

Table 12. Change checklist by area.

Table 13. Glossary of terms used in this guide.

Abstract

This guide documents the architecture, setup procedure, integration points, testing approach, deployment considerations and maintenance practices for NexClinic, a web based outpatient clinic, appointment, communication and healthcare administration system. It describes a Next.js and TypeScript frontend, a Django and Django REST Framework backend, a Django Channels real time layer for chat and live notifications and Celery and Redis for background processing, backed by a PostgreSQL compatible database. Exact field names, endpoint paths and directory structures are described as representative of a suitable implementation rather than asserted as literal facts about a specific codebase and are flagged accordingly throughout. Known limitations, including the simulated payment workflow and the ASGI requirement for real time features are stated explicitly. The guide is intended to be read alongside the companion NexClinic User Manual, which documents day to day operation from each user role's perspective.

Index Terms

Web application architecture, Django REST Framework, Django Channels, JSON Web Token authentication, WebSocket communication, Celery background tasks, Healthcare information system, Role based authorization.

I. System Overview

This guide describes NexClinic from the perspective of a developer, maintainer, tester, reviewer or deployment operator. It follows the numbered heading, figure and table conventions associated with IEEE technical documentation [1], produced in the spirit of the guidance issued through the IEEE Author Center [2], adapted here for a university software project rather than a journal submission. It explains the architecture, setup, integration points, testing approach, deployment considerations and maintenance practices for the system. It assumes familiarity with general web development concepts and does not repeat basic tutorials for the underlying frameworks; where such material is useful, it is referenced rather than reproduced [9], [14].

Business Purpose

NexClinic exists to connect patients, doctors and hospital administrators in a single outpatient clinic workflow: doctor discovery, appointment scheduling, clinical documentation and hospital-level administration, supported by account security and communication features that keep every role informed.

Major User Roles

The system recognizes four roles: patient, doctor, hospital administrator and system administrator, described functionally in Section II of the companion User Manual. Each role is represented in the backend by role information attached to a user account and is reflected in the frontend by a distinct set of protected routes.

Major Functional Areas

- Identity and account management, including OTP-based registration and login verification.
- Doctor discovery and hospital association.
- Appointment scheduling, covering slot creation, booking, state transitions, and clinical-record entry.
- Medication tracking, including reminders and logs.
- Real-time advice chat between patients and doctors.
- Notifications, both in-application and by email, including hospital announcements.
- Hospital administration, including doctor verification, schedule templates, and reporting.

High Level Architecture

NexClinic follows a conventional separated frontend/backend architecture. The frontend is a Next.js application that renders the user interface and communicates with the backend over HTTPS and, in the case of chat and live notifications, over WebSocket connections. The backend is a Django project exposing a REST API through Django REST Framework, together with a Django Channels layer for WebSocket traffic. Background processing, such as reminder delivery, runs through Celery workers coordinated by a Redis-compatible broker. Persistent data is stored in a PostgreSQL-compatible relational database.

Browser (Next.js UI)	Frontend API routes	Django REST Framework	PostgreSQL database
----------------------	---------------------	-----------------------	---------------------

Fig. 1. High-level request path from browser to backend services

Main Data Flow

A typical request originates in the browser, is sent to a Next.js API route acting as a thin proxy, and is forwarded to the Django REST Framework backend, which applies authentication and permission checks before reading from or writing to the database. Real-time features bypass this HTTP path for their live component and instead maintain an authenticated WebSocket connection managed by Django Channels; background tasks are queued separately through Celery and Redis rather than being processed inline with a web request.

II. Architecture

This section describes each architectural layer in turn and then shows how they combine for three representative flows: a normal API request, a WebSocket chat session, and a background notification.

Frontend

The frontend is built with Next.js using the App Router, React, and TypeScript, styled with Tailwind CSS [3], [4], [6], [8]. It is organized around role-specific route groups so that a patient, doctor, hospital administrator, and system administrator each land on a distinct set of pages after authentication.

Backend

The backend is a Django project using Django REST Framework to expose a JSON API [9], [14]. Django applications are organized by domain area: users, patients, doctors, hospitals, appointments, chat and notifications, each contributing models, serializers, views and URL configuration for its area of responsibility.

Database

NexClinic targets a PostgreSQL-compatible relational database [23], accessed through Django's object-relational mapper and managed through Django's migration system, which is described further in Section V.

Real-Time Services

Django Channels extends the Django application to handle WebSocket connections alongside ordinary HTTP requests, using an ASGI server rather than the traditional WSGI interface [18]. This layer supports the advice chat feature and live notification delivery described in Sections XI and XII.

Background Services

Celery workers process background and scheduled work, such as sending reminder notifications, using a Redis-compatible broker to queue tasks and, optionally, to store task results [21], [22].

Email Service

Outbound email, including registration and login OTP codes, password reset links and selected notifications, is sent through a configured SMTP service. In local development, this is typically replaced with a console or file based backend so that OTP codes can be read directly from logs rather than requiring a live mail server.

File Storage

Uploaded files, including medical documents, chat attachments, announcement attachments and profile images, are written to a configured media storage location. In production this should be persistent and, where the deployment target supports it, separated from the application server's local disk so that files survive redeployment.

Component Architecture Diagram

Browser: Next.js frontend (React, TypeScript, Tailwind CSS)		
Frontend API routes (backend for frontend proxy)		
Django REST Framework API		Django Channels (WebSocket)
PostgreSQL database	Celery workers + Redis broker	SMTP email service

Fig. 2. System component architecture (black and white schematic)

Request Flow

A standard HTTP request flow is: the browser calls a frontend API route; the frontend route attaches the stored authentication cookie and forwards the request to the Django REST Framework backend; the backend authenticates the request, applies permission and object-level checks, executes the corresponding view logic against the database, and returns a JSON response; the frontend route relays that response back to the browser for rendering.

WebSocket Flow

A WebSocket flow begins when the browser opens a connection to the Channels-managed WebSocket endpoint, presenting its authentication token during the connection handshake. Once authenticated, the connection is associated with a channel group representing the relevant chat thread or notification stream, and messages published to that group are pushed to every connected participant without a page reload.

Notification Flow

An event that should notify a user, such as an appointment update, is recorded as a notification and, where real-time delivery is configured, published to the affected user's notification channel over WebSockets. Where email delivery applies, a Celery task sends the corresponding message asynchronously so that the triggering request is not held up waiting on SMTP.

III. Technology Stack

Table 4 summarizes the technologies referenced throughout this guide, organized by architectural layer.

Layer	Technology	Purpose
Frontend framework	Next.js (App Router)	Server-rendered and client-rendered React pages, route protection, and API route handlers [3], [4].

Layer	Technology	Purpose
Frontend language	TypeScript	Static typing across components, API route handlers, and response models [7].
Frontend styling	Tailwind CSS	Utility-first styling for responsive, consistent layouts [8].
Frontend library	React	Component-based rendering of the user interface [6].
Backend framework	Django	Application structure, ORM, migrations, and the built-in administration interface [9].
API layer	Django REST Framework	Serialization, viewsets, and permission handling for the REST API [14].
Authentication	JWT via Simple JWT	Stateless access and refresh tokens for authenticated sessions [17].
Real-time layer	Django Channels	WebSocket support for chat and live notifications, served over ASGI [18].
Background processing	Celery	Asynchronous and scheduled tasks such as reminders and email delivery [21].
Message broker	Redis (or Redis-compatible service)	Broker and result backend for Celery, and channel layer backing for Channels [22].
Database	PostgreSQL (or PostgreSQL-compatible service)	Primary relational data store [23].
Email delivery	SMTP service	Delivery of OTP codes, password-reset links, and email notifications.
Media storage	Persistent file storage	Storage of uploaded medical documents, chat attachments, and images.

Table 4. Technology stack by architectural layer

IV. Suggested Project Organization

The following structure describes a suitable way to organize the NexClinic monorepo. Exact directory and application names may differ in a given implementation; developers should treat this section as an orientation guide rather than an exact directory listing.

Top-Level Directory	Contents
backend/	The Django project: settings, URL configuration, ASGI/WSGI entry points, and one Django application per domain area.
frontend/	The Next.js application: route groups by role, shared components, API route handlers, and TypeScript types.
documentation/	Project documentation, including this guide and the companion User Manual.
media/	Local development location for uploaded files when cloud storage is not configured.

Top-Level Directory	Contents
deployment/	Deployment configuration, such as process definitions for ASGI serving, and environment-variable examples.
tests/	Backend and frontend automated tests, or tests colocated within backend/ and frontend/ respectively.

Table 5. Suggested top-level project organization

Within the backend, representative Django applications correspond to the functional areas introduced in Section I: users (accounts, OTP and authentication), patients (profile, medical history and medication), doctors (profile, slots and clinical records), hospitals (hospital profile, doctor associations, schedule templates and announcements), appointments (booking and state transitions), chat (advice chat threads and messages) and notifications (notification records and delivery).

V. Local Development Setup

The commands in this section are examples. Adjust package names, versions, and paths to match the actual implementation before running them.

Required Software

- Python 3.x, for the Django backend.
- Node.js and a package manager such as npm, for the Next.js frontend.
- A PostgreSQL-compatible database server, or a development-only fallback such as SQLite where supported.
- A Redis-compatible server, for Celery and Channels.
- Git, for version control.

Backend Setup

1. Create and activate a Python virtual environment with `python m venv venv`, then activate it for your shell.
2. Install backend dependencies with `pip install r requirements.txt` (example command; adjust to the actual requirements file).
3. Copy the environment variable example file to a local `.env` file and fill in development values, as described in Section VI.
4. Apply database migrations: `python manage.py migrate`.
5. Create an administrator account: `python manage.py createsuperuser`.
6. Start the development server: `python manage.py runserver`, or an ASGI-capable equivalent if WebSocket features are needed locally, as covered in Section XVI.

Frontend Setup

1. From the frontend directory, install dependencies: `npm install`.
2. Copy the environment-variable example file to a local `.env.local` file and set the backend and WebSocket URLs, as described in Section VI.
3. Start the development server: `npm run dev`.

4. Open the printed local URL in a browser.

Database Configuration

Local development can use a PostgreSQL instance matching the production target, or a lightweight fallback such as SQLite when the implementation supports it, to reduce setup overhead. Whichever option is used, the connection details are supplied through the `DATABASE_URL` environment variable described in Section VI.

Redis Setup

A local Redis instance, or a hosted development instance, is needed for Celery task queuing and for the Channels layer if it is configured to use Redis as its channel-layer backend. The connection details are supplied through `REDIS_URL` and the Celery-specific variables in Section VI.

Email Development Setup

For local development, configure Django's email backend to write messages to the console or to local files rather than sending them through a live SMTP server, so that OTP codes and reset links can be read directly from the terminal without external dependencies.

Migrations

Run `python manage.py makemigrations` after changing a model, followed by `python manage.py migrate` to apply the change to the local database. The full migration workflow, including production considerations, is covered in Section XVI.

Creating an Administrator Account

Use `python manage.py createsuperuser` to create an account with access to the Django administration interface, used for a portion of system-administrator work as noted in the User Manual.

Starting Local Services

1. Start the database and Redis services (locally installed, or through a container tool such as Docker Compose if the project provides a configuration for one).
2. Start the Django backend, using an ASGI server if WebSocket features are being exercised locally.
3. Start a Celery worker in a separate terminal: `celery A <project_name> worker l info` (example command).
4. Start a Celery beat scheduler if periodic tasks are enabled: `celery A <project_name> beat l info` (example command).
5. Start the Next.js frontend development server.

VI. Environment Variables

Table 5 lists environment variables relevant to NexClinic. Example values are for local development only and must never be reused in a production deployment. Secrets should always be supplied through environment configuration rather than committed to source control.

Variable	Purpose / Example Value / Security Note
DJANGO_SECRET_KEY	Purpose: cryptographic signing key used by Django. Example: a long random string. Security: must be unique per environment and kept out of source control.
DJANGO_DEBUG	Purpose: enables verbose error pages during development. Example: True locally, False in production. Security: must be False in production to avoid leaking internals.
DJANGO_ALLOWED_HOSTS	Purpose: restricts which hostnames may serve the application. Example: localhost,127.0.0.1 locally. Security: must list only the actual production domain(s) in deployment.
DATABASE_URL	Purpose: database connection string. Example: postgres://user:password@localhost:5432/nexclinic. Security: use a dedicated database user with least-privilege access.
CORS_ALLOWED_ORIGINS	Purpose: origins permitted to make cross-origin requests to the API. Example: http://localhost:3000 locally. Security: must be limited to the known frontend origin(s) in production.
CSRF_TRUSTED_ORIGINS	Purpose: origins trusted for cross-site request forgery protection on state-changing requests. Example: http://localhost:3000 locally. Security: keep aligned with CORS_ALLOWED_ORIGINS.
EMAIL_HOST	Purpose: SMTP server hostname. Example: smtp.example.com or a local development value. Security: use provider-supplied credentials, not personal ones.
EMAIL_PORT	Purpose: SMTP server port. Example: 587. Security: prefer a port and configuration that supports TLS.
EMAIL_HOST_USER	Purpose: SMTP authentication username. Example: a service account address. Security: never share this account's password outside the deployment environment.
EMAIL_HOST_PASSWORD	Purpose: SMTP authentication password or application key. Example: not shown. Security: store only as a secret; never commit to source control.
EMAIL_USE_TLS	Purpose: enables TLS for SMTP delivery. Example: True. Security: should remain True outside of trusted local testing.
REDIS_URL	Purpose: connection string for the Redis-compatible service. Example: redis://localhost:6379/0. Security: restrict network access to trusted hosts.
CELERY_BROKER_URL	Purpose: broker connection used by Celery to queue tasks. Example: same as REDIS_URL. Security: as for REDIS_URL.
CELERY_RESULT_BACKEND	Purpose: where Celery stores task results, if used. Example: same as REDIS_URL. Security: as for REDIS_URL.

Variable	Purpose / Example Value / Security Note
NEXT_PUBLIC_BACKEND_URL	Purpose: base URL the frontend uses to reach the backend API. Example: http://localhost:8000 locally. Security: this value is exposed to the browser; do not place secrets in NEXT_PUBLIC_ variables.
NEXT_PUBLIC_WS_URL	Purpose: base URL for WebSocket connections. Example: ws://localhost:8000 locally, wss:// in production. Security: exposed to the browser; use wss:// in production.
Media-storage credentials (when used)	Purpose: access keys for a cloud storage provider hosting uploaded files. Example: provider-specific access key and secret. Security: use a scoped, least-privilege credential and never commit it to source control.

Table 6. Environment variables, purpose, example values, and security considerations

VII. Authentication Architecture

NexClinic authentication combines email-and-password credentials, OTP verification at two points, and JSON Web Tokens for session handling once a user is signed in [11], [17], [25].

Registration and Registration OTP

Registration creates an account in an inactive state and issues an OTP to the supplied email address. Submitting the correct, unexpired OTP activates the account. Resend and expiry controls limit how the OTP mechanism can be abused, and repeated invalid attempts trigger a temporary lockout rather than allowing unlimited guessing [28].

Login and Login OTP

Login validates the submitted email and password against the stored, hashed credential. Where an additional verification step is configured, a second OTP is issued and must be submitted before a full session is established, adding a layer of protection beyond the password alone [28].

JWT Access and Refresh Tokens

Once authenticated, the backend issues a short-lived access token and a longer-lived refresh token, following the JSON Web Token format [25]. The access token authorizes individual API requests; the refresh token is used to obtain a new access token once the current one expires, without requiring the user to re-enter credentials.

HTTP Only Cookies and Token Refresh

Tokens are stored in HTTP only cookies rather than in client side JavaScript accessible storage, reducing exposure to cross site scripting attacks [10], [32]. The frontend's API route layer forwards these cookies to the backend and handles refreshing the access token transparently when it expires during a session.

Logout

Logout clears the authentication cookies on the client and, where supported, invalidates the associated refresh token on the backend so that it cannot be reused after the session ends.

Password Reset

A password reset request generates a secure, time limited, single use link sent to the account's registered email address. Following the link allows the user to set a new password; the link cannot be reused afterward [31].

Role Validation and Authorization

Every authenticated request carries the user's role, which the backend uses to determine what the request is permitted to do. Frontend route guards prevent obviously mismatched navigation, for example, keeping a patient out of the hospital-administrator route group, but the backend's own permission checks are the actual security boundary, described further below.

Frontend Route Guards vs. Backend Permissions

IMPORTANT:

Frontend route protection exists to improve navigation and user experience: it stops a user from wandering into a workspace that does not apply to their role and prevents a confusing interface state. It is not a security control by itself, because a client-side check can always be bypassed by someone interacting with the API directly. Backend permission enforcement, applied on every request by Django REST Framework's permission classes, is the actual security boundary [16].

Object Level Authorization

Beyond role checks, several NexClinic operations require object level authorization, such as confirming that the appointment a doctor is trying to update actually belongs to that doctor, or that the medical record a patient is viewing is their own. This is enforced in view logic rather than relying on role alone, consistent with general authorization guidance [29].

WebSocket Authentication

WebSocket connections used for chat and live notifications are authenticated during the connection handshake using the same JWT credentials as HTTP requests, validated by Channels middleware before the connection is accepted [19], [34].

OTP Expiry and Attempt Controls

Both registration and login OTP codes expire after a limited time and are subject to a maximum number of incorrect attempts before a temporary lockout is applied, reducing the practicality of brute-force guessing against the OTP mechanism.

Authentication Sequence Diagrams

Submit registration	Account created (inactive)	OTP emailed	OTP submitted	Account activated
---------------------	----------------------------	-------------	---------------	-------------------

Fig. 3. Registration and OTP verification sequence

Submit credentials	Password verified	Login OTP (if required)	Access + refresh tokens issued	Cookies set
---------------------------	--------------------------	--------------------------------	---------------------------------------	--------------------

Fig. 4. Login and token creation sequence

Access token expires	Frontend calls refresh endpoint	Refresh token validated	New access token issued
-----------------------------	--	--------------------------------	--------------------------------

Fig. 5. Token refresh sequence

Browser opens WebSocket	Token sent in handshake	Channels middleware validates token	Connection accepted
--------------------------------	--------------------------------	--	----------------------------

Fig. 6. WebSocket authentication sequence

VIII. Domain Model

This section describes NexClinic's conceptual entities and how they relate to one another. Exact field names and database-level constraints depend on the implementation and are not asserted here; developers should confirm specifics against the actual model definitions in the codebase.

Entity	Represents	Key Relationships
User	A single account with an email identity, credential, role, and activation state.	Specializes into a Patient, Doctor, or Hospital Administrator profile depending on role.
Patient	Profile, contact, emergency-contact, and health information for a patient user.	Owns medications, medication reminders and logs, uploaded documents, and appointments.
Doctor	Professional profile, specialization, qualifications, and fee information for a doctor user.	Associated with one or more Hospitals; owns appointment slots and participates in appointments.
Hospital	A clinic or hospital profile administered by one or more Hospital Administrators.	Associated with Doctors; owns schedule templates, announcements, and hospital-level appointment oversight.
Doctor and Hospital Relationship	The association between a Doctor and a Hospital, including verification status and the fee at that hospital.	Connects Doctor and Hospital; gates whether the doctor is visible for booking at that hospital.
Appointment Slot	A specific date, time, hospital, and capacity at which a Doctor is available.	Created directly by a Doctor or generated from a Schedule Template; consumed by Appointments.
Appointment	A booking made by a Patient against an Appointment Slot, tracked through a defined set of states.	Links Patient, Doctor, and Appointment Slot; may produce a Medical Record and Prescription.

Entity	Represents	Key Relationships
Medical Record	Consultation notes and diagnosis entered by a Doctor for a completed or in-progress Appointment.	Belongs to an Appointment and to the associated Patient's history.
Prescription	Medication, dosage, and instructions issued by a Doctor for a Patient.	Typically linked to an Appointment and visible in the Patient's medical history.
Medication	A medication entry maintained by a Patient, whether prescribed through NexClinic or recorded independently.	Owned by a Patient; may have associated Reminders and Logs.
Medication Reminder	A configured time at which a Patient should be reminded to take a Medication.	Belongs to a Medication; triggers Notifications through background tasks.
Medication Log	A record of a medication event, such as a dose taken or skipped.	Belongs to a Medication and a Patient.
Advice Availability	A Doctor's current willingness to accept new advice-chat conversations.	Set by the Doctor; gates whether a Patient can start a new Chat Thread.
Chat Thread	An advice-chat conversation between a Patient and a Doctor.	Has participants (Patient, Doctor), Messages, and a status such as open or closed.
Chat Message	An individual message, and optional attachment, within a Chat Thread.	Belongs to a Chat Thread; has a sender and a read state.
Notification	A record informing a User of a relevant event.	Belongs to a User; may originate from an Appointment, Announcement, or other event.
Announcement	A message published by a Hospital Administrator to a targeted audience.	Belongs to a Hospital; generates Notifications for the targeted Users.
Activity Log	A record of a significant action taken within the system, such as a verification decision or cancellation.	References the acting User and the affected entity.

Table 7. Conceptual domain entities and their relationships

Conceptual Entity Relationship Diagram

User			
Patient	Doctor		Hospital Administrator
Appointment Slot	Appointment		Chat Thread
Medical Record	Prescription	Notification	Activity Log

Fig. 7. Conceptual entity relationship overview (simplified, black and white)

The diagram simplifies the full set of relationships for readability. In particular, the Doctor-Hospital relationship, medication-related entities, and announcement targeting are described in the preceding table rather than shown as separate boxes.

IX. REST API Design

NexClinic exposes its backend functionality through a REST API built with Django REST Framework [14]. Exact endpoint paths, field names, and pagination conventions depend on the final implementation; the groups below describe representative, expected behavior rather than a literal endpoint reference.

Endpoint Group	Purpose / Typical Operations	Authentication and Authorization	Typical Response / Common Errors
/api/users/	Registration,OTP verification,login, token refresh, logout, and password reset.	Most endpoints are unauthenticated by design, since they establish or recover a session; rate limiting and OTP attempt controls apply.	JSON account objects or confirmation messages. Errors: 400 validation, 401 invalid credentials, 429 if rate-limited.
/api/patients/	Patient profile, medical history, medication, reminders and logs, and medical-document uploads.	Requires an authenticated Patient for write operations; a patient may access only their own records, a doctor only through a legitimate appointment relationship.	JSON profile and list data; multipart for uploads. Errors: 401 unauthenticated, 403 cross-account access, 404 missing record.
/api/doctors/	Doctor profile, specialization and qualification management, hospital-association requests, and slot management.	Requires an authenticated Doctor for write operations; a doctor manages only their own profile and slots, gated by hospital verification.	JSON doctor profile and slot lists. Errors: 401 unauthenticated, 403 cross-account modification, 409 slot conflict.
/api/hospitals/	Hospital profile, doctor-association review, schedule templates, slot generation, announcements, and reports.	Requires an authenticated Hospital Administrator, restricted to the hospital(s) they administer.	JSON hospital, template, and report data. Errors: 401 unauthenticated, 403 cross-hospital access, 400 invalid template.
/api/appointments/	Booking, status transitions (accept, reject, propose new time, complete, cancel), and clinical-record entry.	Requires an authenticated Patient, Doctor, or Hospital Administrator; transitions restricted to the role authorized for that transition and the specific participants.	JSON appointment state. Errors: 400 invalid transition, 403 unauthorized actor, 409 booking conflict.

Endpoint Group	Purpose / Typical Operations	Authentication and Authorization	Typical Response / Common Errors
/api/chat/	Advice-chat thread listing, message history, and attachment upload; live delivery uses WebSockets, not this REST group.	Requires an authenticated participant of the thread; a doctor must have advice chat marked available to accept a new thread.	JSON thread and message history. Errors: 401 unauthenticated, 403 non-participant, 404 missing thread.
/api/notifications/	Notification history, read-state updates, and notification-preference management.	Requires an authenticated user of any role, restricted to their own notifications and preferences.	JSON notification lists and preferences. Errors: 401 unauthenticated, 404 missing notification.

Table 8. Representative REST API endpoint groups

Exact endpoint paths, field names, and versioning conventions depend on the final implementation. Developers should confirm the authoritative list against the backend's URL configuration rather than this summary alone. Regardless of the exact path structure, resource-oriented endpoints and standard HTTP status codes are used consistently with general HTTP semantics [24], and authentication classes are configured through Django REST Framework's own authentication layer [15].

X. Appointment and Scheduling Logic

Slot Creation: Direct and Template Generated

An appointment slot can be created directly by a doctor, specifying hospital, date, start and end time, and capacity, or generated in bulk by applying a hospital administrator's schedule template over a chosen date range. Both paths converge on the same underlying slot representation, so booking logic does not need to distinguish how a slot was created.

Date and Time Validation

Slot creation and generation validate that the start time precedes the end time, that the date is not in the past, and that the resulting slot does not conflict with an existing slot for the same doctor and hospital in a way the system does not support (for example, two overlapping slots at full capacity). Template-based generation applies the same validation to every slot it produces.

Hospital and Doctor Relationships

A slot can only be created for a hospital the doctor has a verified association with, which prevents a doctor from appearing to offer availability at a hospital that has not approved them.

Capacity

Each slot carries a capacity value. A booking is only accepted while the number of existing bookings against the slot is below its capacity; once capacity is reached, the slot is shown as full and is no longer offered for new bookings.

Booking Transactions and Duplicate Booking Prevention

Booking an appointment is performed as a single database transaction: the slot's current booking count is checked and the new appointment record is created atomically, so that two near simultaneous booking requests cannot both succeed against a slot that only has capacity for one of them. This is a standard use of database level transactional guarantees to avoid a race condition between the capacity check and the write.

Appointment States

Table 6 lists the appointment states referenced throughout this guide and the companion User Manual, together with the role responsible for each transition.

From State	Action	Actor	To State
(new booking)	Patient submits a booking against an available slot.	Patient	Pending
Pending	Doctor accepts the booking.	Doctor	Accepted
Pending	Doctor rejects the booking.	Doctor	Rejected
Pending	Doctor proposes a different date or time.	Doctor	Reschedule Pending
Reschedule Pending	Patient accepts the proposed time.	Patient	Accepted
Reschedule Pending	Patient declines the proposed time.	Patient	Cancelled
Accepted	Patient cancels before the appointment.	Patient	Cancelled
Accepted	Hospital administrator cancels with a reason.	Hospital Administrator	Cancelled
Accepted	Doctor marks the consultation as complete.	Doctor	Completed

Table 9. Appointment state transitions and responsible actors

Clinical Record Creation

Consultation notes, diagnosis, and prescription entries are associated with a specific appointment and are normally entered once that appointment reaches the Completed state, or during the consultation immediately before completion, depending on the doctor's workflow.

Auditability

State transitions and cancellation actions are significant enough to be recorded in the activity-log mechanism described in Sections XII and the User Manual, supporting later review of who took a given action and why.

Time Zone Testing

Because appointment scheduling is inherently date and time sensitive, testing should specifically exercise behavior around time zone handling. For example, testers should confirm that a slot's displayed time is consistent for a patient and doctor who may be viewing the application from

different configured time zones, and that stored timestamps are compared consistently regardless of the server's local time zone setting.

Concurrent Booking Risks

Beyond the transactional safeguard described above, developers extending the booking logic should be careful not to introduce a check then write pattern that reads capacity, performs unrelated work and only later writes the new booking, since that reintroduces the race condition the transaction is designed to prevent.

XI. Real-Time Chat

Advice chat is implemented using Django Channels over WebSockets, layered on top of Django's existing HTTP-based authentication model [18], [20].

WebSocket Connections and Authentication

A chat client opens a WebSocket connection to a Channels-managed endpoint and presents its JWT access token during the connection handshake. Channels middleware validates the token before allowing the connection to proceed, consistent with the WebSocket authentication sequence described in Section VII [19].

Eligible Chat Threads and Participants

A chat thread exists between exactly one patient and one doctor, and is only eligible to be created or joined when the underlying condition NexClinic requires, such as an existing or recent appointment relationship and the doctor's advice-chat availability being turned on, is satisfied.

Message Delivery and Attachments

A sent message is written to the database and published to the thread's channel group so that every connected participant receives it immediately; a participant who is not currently connected will see the message the next time they open the thread, since it is also retrievable through the REST chat-history endpoint described in Section IX.

Read State

Each message tracks whether it has been read by the recipient, supporting an unread-count indicator in the chat workspace and notification centre.

Thread Closing

A doctor can close a thread once the matter has been addressed. A closed thread remains readable by both participants but no longer accepts new messages, consistent with the behavior described in the User Manual.

Error Handling and Reconnection

A dropped WebSocket connection does not lose previously sent messages, since they are persisted independently of the live channel; the frontend is expected to detect a dropped connection and attempt to reconnect, falling back to the REST history endpoint to fill in anything missed while disconnected.

Allowed Origins, Hosts, and Production WSS

As with HTTP traffic, WebSocket connections are restricted to allowed hosts and origins configured for the deployment. In production, WebSocket connections must use the encrypted wss:// scheme rather than the plain ws:// scheme used in local development [34].

XII. Notifications and Background Tasks

Notification History and Read State

Every notification generated for a user is persisted, giving a complete history independent of whether real-time delivery succeeded, and each notification tracks whether it has been read, mirroring the behavior described for chat messages.

Real Time and Email Delivery

Where a live WebSocket connection is available, a new notification is pushed to the user immediately. Where email delivery is configured for a given notification category, delivery is handled by a Celery task so that the request that triggered the notification is not delayed waiting for the SMTP round trip.

Announcements, Target Audiences, and Preferences

A hospital administrator's announcement is expanded into individual notification records for every user in the targeted audience, for example, all patients, all doctors or doctors within a specific specialization, at the time the announcement is published. A user's notification preferences can narrow which categories they receive, but do not override notifications required for account security.

Celery Tasks, Redis, and Worker Processes

Background work, including reminder delivery, announcement fan out and email sending, runs as Celery tasks, queued through a Redis compatible broker and processed by one or more worker processes running separately from the web server process [21], [22].

Periodic Tasks and Retry Behavior

Recurring work, such as checking for medication reminders that are due, is handled through Celery's periodic-task scheduling (commonly Celery Beat) where the deployment enables it. Individual tasks should be written to retry on transient failure, such as a temporary SMTP outage, rather than silently dropping the notification.

Operational Logging

Task execution, including failures, should be logged so that a missed or delayed notification can be traced back to its cause, whether that is a broker outage, a task exception or an email-delivery failure, consistent with general logging guidance [35].

Notification Flow Diagram

Triggering event occurs	Notification record created	Push over WebSocket	Celery task (email, if applicable)	User sees notification
-------------------------	-----------------------------	---------------------	------------------------------------	------------------------

Fig. 8. Notification generation and delivery flow

XIII. File Uploads

NexClinic accepts uploaded files in several contexts: patient profile images, medical documents, chat attachments, and announcement attachments. Each context applies the same general controls, described below [12], [33].

File Type and File Size Validation

Uploads are restricted to a defined set of accepted file types appropriate to the context (for example, common image and document formats for medical documents) and to a maximum file size, both of which are validated on the backend rather than relying solely on frontend checks, since a frontend only check can be bypassed by a direct API request.

Authorization

An upload is associated with the uploading user and, where relevant, with the specific record it belongs to (a chat thread, an appointment, or a patient's own document list), and access to retrieve it later is subject to the same authorization rules described in Section VII.

Secure File Names and Malware Considerations

Uploaded files should be stored under generated, non guessable identifiers rather than the original file name, to avoid path traversal or collision issues and to reduce the value of guessing at file locations directly. Production deployments handling real clinical documents should also consider malware scanning of uploaded content before it is made available to other users, which is not assumed to be implemented by default.

Persistent Storage and Signed URLs

In production, uploaded files should be written to persistent, durable storage rather than the application server's local disk, since local disk storage is lost on redeployment in many hosting environments. Where a cloud storage provider is used, serving files through signed, time-limited URLs is preferable to making the storage bucket directly public.

Privacy, Retention, and Deletion

Because uploaded content can include sensitive clinical information, retention and deletion practices for uploaded files should be defined deliberately as part of operating NexClinic in a real clinical setting, rather than left to the storage provider's defaults.

XIV. Frontend Development

App Router, Public and Protected Routes

The frontend uses the Next.js App Router to organize pages into route groups [4]. Public routes cover registration, login, and OTP verification; protected routes require a valid session and are further separated by role so that, for example, doctor-specific pages are not reachable by a patient session.

Role Specific Workspaces and Shared Components

Each role's protected route group renders its own dashboard and feature pages, while sharing common building blocks such as buttons, form fields, tables, status badges and layout shells through a shared component library, keeping the visual language consistent across roles as described in the User Manual's navigation conventions.

API Route Handlers and the Backend for Frontend Pattern

Rather than calling the Django backend directly from browser side code, the frontend routes requests through its own Next.js API route handlers, which attach the stored authentication cookie, forward the request to the backend and relay the response. This backend for frontend pattern keeps the browser from needing direct knowledge of backend authentication details and gives the frontend a place to adapt or aggregate responses if needed [5].

Secure Cookie Forwarding

Authentication cookies are handled as HTTP-only and are forwarded by the frontend's own server-side code rather than being read or set directly by browser-side JavaScript, consistent with the authentication architecture in Section VII.

TypeScript Response Models

Backend responses are described using TypeScript types or interfaces on the frontend, so that a mismatch between what the backend returns and what a component expects is caught during development rather than at runtime [7].

Forms, Validation, and Interface States

Forms validate input on the frontend for immediate feedback and rely on the backend's own validation as the authoritative check. Pages are expected to handle loading, empty, error, unauthorized, and expired-session states explicitly, rather than leaving the interface in an ambiguous state while data is fetched or when a request fails.

Responsive Layouts and Accessibility

Tailwind CSS utility classes are used to build layouts that adapt across desktop and mobile viewport sizes [8]. Interface elements follow general accessibility practice, including labeled form fields, sufficient contrast and keyboard-operable controls, expanded on in Section XVIII [36].

XV. Testing and Verification

Table 7 sets out a verification matrix covering the test levels relevant to NexClinic, a representative command or method for each, a coverage goal, and the expected result. Commands are examples and should be adapted to the project's actual test configuration.

Test Level	Recommended Command / Method	Coverage Goal	Expected Result
Backend unit tests	python manage.py test (example command)	Model methods, serializers, and isolated view logic.	All tests pass; failures identify the specific broken unit.
Backend integration tests	python manage.py test, targeting multi-step flows	End-to-end backend behavior such as registration through OTP activation.	The full flow completes and produces the expected database state.

Test Level	Recommended Command / Method	Coverage Goal	Expected Result
Permission tests	Role-scoped API test cases	Every endpoint enforces the correct role and object-level access.	Unauthorized requests are rejected with 401 or 403 as appropriate.
Appointment-state tests	State-transition test cases per role	Every transition in Table 6 is exercised, including disallowed transitions.	Allowed transitions succeed; disallowed transitions are rejected.
Concurrent-booking tests	Simulated simultaneous requests against one slot	The booking transaction correctly prevents over-booking a slot.	Only as many bookings succeed as the slot's capacity allows.
OTP tests	Expiry and attempt-limit test cases	OTP issuance, resend, expiry, and lockout behavior.	Expired or over-attempted codes are rejected; valid codes succeed.
WebSocket tests	Channels testing utilities (example: Channels' test consumer)	Connection authentication and message delivery within a thread.	Unauthenticated connections are refused; authenticated messages are delivered to participants.
Notification tests	Task-triggered notification test cases	Notification creation and, where applicable, delivery task execution.	A triggering event produces the expected notification record.
File-upload tests	Type and size validation test cases	Rejection of disallowed file types and oversized files.	Invalid uploads are rejected with a clear error; valid uploads succeed.
Frontend linting	npm run lint (example command)	Code style and common error patterns across the frontend codebase.	No lint errors reported.
Frontend build	npm run build (example command)	The frontend compiles and type-checks successfully for production.	The build completes without TypeScript or build errors.
Component tests	A frontend component-testing tool configured for the project	Individual UI components render and behave correctly in isolation.	Component tests pass for representative props and states.
End-to-end tests	A browser-automation tool configured for the project	Full user journeys, such as booking an appointment from login to confirmation.	The journey completes without unexpected errors.
Manual smoke tests	Walkthrough of core flows after a deployment or major change	Confidence that core flows work in the actual running environment.	No blocking issues found in the core flows checked.

Test Level	Recommended Command / Method	Coverage Goal	Expected Result
Security tests	Manual or tool-assisted review against OWASP guidance	Common web-application weaknesses across authentication, authorization, and input handling.	No high-severity findings outstanding [27].
Accessibility tests	Manual review and, where available, an automated accessibility checker	Conformance with basic WCAG expectations for forms and navigation.	No critical accessibility issues outstanding [36].

Table 10. Verification matrix by test level

XVI. Deployment

Production Environment Variables

Every variable listed in Section VI must be set with production-appropriate values before deployment, with `DJANGO_DEBUG` explicitly set to `False` and `DJANGO_ALLOWED_HOSTS`, `CORS_ALLOWED_ORIGINS`, and `CSRF_TRUSTED_ORIGINS` restricted to the actual production domain(s) [10], [13].

Migrations, Static Files, and Persistent Media

Database migrations must be applied as part of each deployment (`python manage.py migrate`), static assets collected for serving (`python manage.py collectstatic`, where applicable), and media storage configured to persist across deployments rather than relying on local, ephemeral disk.

HTTPS and WSS

Production traffic must be served over HTTPS, and WebSocket connections over the corresponding encrypted `wss://` scheme, both to protect credentials and tokens in transit and because browsers restrict certain features to secure contexts.

Allowed Hosts, CORS, and CSRF

These settings, introduced in Section VI, must be reviewed specifically for the production domain before launch; overly permissive values here weaken the platform's security posture significantly.

Secure Cookies

Authentication cookies should be marked `Secure` so that browsers only transmit them over HTTPS, in addition to being `HTTP-only` as described in Section VII.

ASGI Deployment and WSGI Limitations

IMPORTANT:

A WSGI only deployment cannot serve Django Channels WebSocket connections, because WSGI is a synchronous, request response interface with no concept of a long lived bidirectional connection. Any deployment that needs advice chat or live notification delivery to work must run the application behind an ASGI capable server process, with WebSocket traffic routed to the Channels application rather than the plain WSGI application [18].

Redis, Celery Workers, and Scheduler

A production Redis-compatible service must be reachable by both the web process and the Celery worker process(es); at least one worker process must be running for background tasks to be processed, and a scheduler process (such as Celery Beat) must run if periodic tasks, including reminder checks, are enabled.

SMTP

A production SMTP provider must be configured with TLS enabled, replacing the console or file based backend used in local development.

Logging and Health Checks

Application logging should be configured to capture errors and significant events at a level suitable for production (avoiding the verbose debug output appropriate only for local development), and a health check endpoint or equivalent mechanism should be available so that the deployment platform can detect a failed instance.

Backups and Rollback Planning

Regular database backups and a documented rollback procedure for both application code and database migrations should be in place before NexClinic is used with real data, since a failed migration or a faulty release is otherwise difficult to recover from cleanly.

Deployment Architecture Diagram

Browser (HTTPS / WSS)			
Next.js production build (frontend host)			
ASGI server: Django + Channels (HTTP and WebSocket)			
PostgreSQL	Redis (broker + channel layer)	Celery worker / scheduler	SMTP provider

Fig. 9. Deployment architecture overview (black and white schematic)

XVII. Security and Privacy

- Secret management: store `DJANGO_SECRET_KEY`, database credentials, email credentials, and storage credentials only as environment-level secrets, never committed to source control.

- Password security: store passwords using Django's built-in password hashing rather than any custom scheme [30].
- OTP protection: enforce expiry and attempt limits on every OTP, as described in Section VII.
- Token security: keep JWT access tokens short-lived and refresh tokens appropriately scoped, and invalidate refresh tokens on logout where supported [17], [25], consistent with current best-practice guidance on token handling [26].
- Secure cookies: HTTP-only and Secure flags on authentication cookies, as described in Sections VII and XVI.
- Least privilege: grant each role, and each database or service credential, only the access it actually needs.
- Backend permission enforcement and object-level authorization: as described in Section VII, treated as the real security boundary rather than frontend route guards [16], [29].
- CORS, CSRF, and allowed hosts: configured restrictively for the actual production origins, as described in Section XVI [10].
- HTTPS and WSS: required in production for all traffic, including WebSocket connections.
- File-upload controls: type and size validation, secure storage naming, and consideration of malware scanning, as described in Section XIII [33].
- Logging controls: capture security-relevant events without logging sensitive values such as raw passwords or OTP codes [35].
- Sensitive-data minimization: collect and retain only the clinical and personal information genuinely needed for NexClinic's purpose.
- Data retention: define and apply retention and deletion practices for accounts, clinical records, chat history, and activity logs.
- Dependency updates: keep backend and frontend dependencies current to receive security fixes from upstream projects.
- Security monitoring and incident response: define a process for detecting and responding to a suspected security incident before NexClinic is used with real patient data.
- Backup protection: secure database backups with the same care as the live database, since a backup is an equally sensitive copy of the same data.

These practices draw on general, widely used security guidance for web applications [27], and should be revisited specifically for a healthcare context before any production use, as noted in the Known Limitations section.

XVIII. Accessibility and Usability

Accessibility is treated as a development concern rather than an afterthought, following general guidance from the Web Content Accessibility Guidelines and the associated forms tutorial [36], [37].

- Semantic headings: pages use a logical heading structure so that assistive technology can navigate content by section.
- Keyboard navigation: interactive elements are reachable and operable using the keyboard alone, without requiring a mouse.
- Form labels: every form field carries a visible, programmatically associated label, rather than relying on placeholder text alone.

- Clear validation errors: an invalid field states what is wrong and, where possible, how to correct it, rather than a generic failure message.
- Sufficient contrast: text and interactive elements meet reasonable contrast expectations against their background.
- Responsive layouts: pages remain usable across common desktop and mobile viewport sizes.
- Accessible tables: data tables use appropriate header markup so that column and row relationships are conveyed to assistive technology.
- Focus visibility: the currently focused element is visually distinguishable when navigating by keyboard.
- Screen-reader support: dynamic updates, such as a new chat message or notification, are announced in a way a screen-reader user can perceive, not only through a visual change.
- Understandable status messages: confirmation and error messages use plain language rather than internal error codes.

XIX. Maintenance and Troubleshooting

Table 8 lists operational problems a developer or operator may encounter while running or maintaining NexClinic, with a likely cause and a recommended action for each.

Problem	Likely Cause	Recommended Action
Frontend cannot reach backend	Incorrect <code>NEXT_PUBLIC_BACKEND_URL</code> , or the backend process is not running.	Confirm the backend is running and reachable, and verify the configured URL matches its actual address.
Authentication loop	A cookie is not being set or forwarded correctly, or the refresh-token flow is failing silently.	Inspect cookie settings and the frontend's token-refresh handling; check backend logs for refresh-endpoint errors.
OTP email failure	SMTP misconfiguration or provider outage.	Check <code>EMAIL_*</code> variables and provider status; confirm the local development console/file backend is not left active in production by mistake.
Database connection failure	Incorrect <code>DATABASE_URL</code> , database service not running, or network/firewall restriction.	Verify the connection string, confirm the database service is reachable, and check firewall or security-group rules.
Migration failure	A conflicting or malformed migration, or an attempt to apply migrations out of order.	Review the migration history, resolve conflicts, and consider a staging-environment rehearsal before reapplying in production.
WebSocket failure	A WSGI-only deployment serving Channels traffic, or a misconfigured channel-layer backend.	Confirm the deployment uses an ASGI-capable server for WebSocket routes, and verify the Redis-backed channel layer is reachable.

Problem	Likely Cause	Recommended Action
Redis failure	The Redis-compatible service is down or unreachable.	Check REDIS_URL connectivity and the service's own health status; Celery and Channels both depend on it.
Celery task failure	An unhandled exception in a task, or the worker process is not running.	Check worker logs for the failing task, and confirm at least one worker process is active and connected to the broker.
Notification delay	A backlog in the task queue, or a slow or failing email provider.	Check queue length and worker throughput, and confirm the email provider is not rate-limiting or rejecting messages.
Media URL failure	Incorrect storage configuration, or a URL that has expired if signed URLs are used.	Verify storage credentials and bucket configuration, and confirm signed-URL expiry settings are appropriate for how files are used.
Missing uploaded files	Files stored on ephemeral local disk in an environment that does not persist it across deployments.	Move media storage to a persistent, durable location before further deployments.
CORS failure	CORS_ALLOWED_ORIGINS does not include the actual frontend origin.	Add the correct origin, matching scheme and port exactly, to the allowed-origins setting.
CSRF failure	CSRF_TRUSTED_ORIGINS does not include the frontend origin, or a cookie/header mismatch.	Align CSRF_TRUSTED_ORIGINS with the frontend origin and confirm the frontend sends the expected CSRF header where required.
Expired cookies	Access or refresh token lifetimes have elapsed without a successful refresh.	Sign in again; if this happens unexpectedly often, review token lifetime settings and the refresh flow.
Incorrect user role	The account was created with the wrong role, or a role change was not applied correctly.	Correct the role through the administrative interface, and confirm the user signs out and back in to refresh their session.
Old activity-log accumulation	No retention or archiving policy has been applied to activity logs.	Define and apply a retention or archiving policy consistent with the project's data-retention decisions.
Failed production build	A TypeScript type error, a missing dependency, or an environment variable required at build time that is unset.	Review the build output for the specific failure and confirm all build-time environment variables are set.

Problem	Likely Cause	Recommended Action
ASGI configuration failure	The ASGI application is not correctly referenced by the server process, or routing between HTTP and WebSocket paths is misconfigured.	Review the ASGI entry point and routing configuration against the Channels documentation [18].

Table 11. Maintenance and troubleshooting reference

XX. Change Checklist

Table 9 gives a checklist to work through before merging or deploying a change that touches the areas listed. Not every item applies to every change; the checklist is a prompt for what to consider rather than a mandatory sequence.

Change Area	Before Merging or Deploying
Model changes	Confirm the change is backward-compatible where possible, and that dependent serializers and views are updated to match.
Migrations	Generate and review the migration, test it against a representative copy of production-like data where feasible, and confirm it is reversible or that a rollback plan exists.
Serializer changes	Confirm field additions or removals do not silently break frontend consumers; update TypeScript response models to match.
API changes	Update the representative endpoint documentation in Section IX, and confirm existing clients are not broken by the change.
Permission changes	Add or update permission tests confirming the new behavior, including object-level checks where relevant.
Frontend type changes	Confirm the frontend build succeeds and that dependent components handle the updated type correctly.
Environment changes	Update the environment-variable table in Section VI and the example environment files, and communicate the change to other developers and operators.
WebSocket changes	Confirm authentication still applies correctly, and that the channel-layer configuration matches the change.
Celery-task changes	Confirm retry behavior is appropriate, and that the task remains idempotent where it may be retried.
Tests	Add or update tests covering the change, and confirm the full relevant test suite passes.
Documentation	Update this guide and the User Manual where the change affects a documented behavior or screen.
Security review	Consider whether the change introduces a new authorization boundary, input source, or data-exposure risk.

Change Area	Before Merging or Deploying
Deployment review	Confirm the change does not require a manual step that is not already captured in the deployment procedure in Section XVI.

Table 12. Change checklist by area

XXI. Known Limitations

The following limitations apply to the current state of NexClinic and are relevant to anyone extending, testing or deploying the system.

- The payment workflow at appointment booking is a mock or simulated gateway and does not represent a real production financial transaction; a certified payment gateway would need to be integrated for genuine clinical use.
- Video consultation is not included; real-time communication is limited to text-based advice chat with supported file attachments.
- Online chat depends on WebSocket and network availability, and an ASGI-capable deployment specifically, as described in Section XVI.
- Real-time notification delivery depends on the configured Channels and Redis infrastructure and may be delayed if that infrastructure is degraded.
- Email OTP and notification delivery depend on SMTP or email-provider configuration and are subject to that provider's own reliability.
- Production file storage requires a persistent, durable configuration; uploaded files are not guaranteed to survive redeployment on ephemeral local disk.
- Some system-administration functions rely on the Django administration interface rather than a purpose-built NexClinic screen, which is functional but not tailored to non-technical administrators.
- Some frontend administration screens may be minimal in their current form.
- NexClinic supports healthcare administration and communication but does not replace professional medical judgment, and is not designed or validated as a clinical decision-support tool.
- NexClinic is not an emergency service platform and should not be relied upon for urgent or emergency communication.
- Production use of NexClinic would require additional security, privacy, clinical, legal, and regulatory review appropriate to the deployment's jurisdiction and setting.
- NexClinic does not claim compliance with HIPAA, GDPR, or any other specific regulation; no such compliance has been formally assessed, and none should be assumed from this documentation.

XXII. Future Improvements

The following are realistic recommendations for future work rather than implemented features. They are included to give a developer picking up the project a sense of reasonable next steps.

- Integrate a real, certified payment gateway in place of the current mock confirmation.
- Add video consultation support alongside the existing text-based advice chat.

- Develop dedicated mobile applications, in addition to the responsive web interface.
- Support calendar integration so patients and doctors can sync appointments with external calendar tools.
- Extend the reporting and analytics area with more advanced, configurable reports.
- Explore health-record interoperability, potentially informed by the HL7 FHIR standard, for exchanging clinical data with other systems [38].
- Improve accessibility further through structured auditing against WCAG success criteria [36].
- Build dedicated audit-review dashboards rather than relying solely on raw activity logs.
- Add multi-factor authentication options beyond the current OTP-based approach.
- Strengthen file-security scanning for uploaded documents and attachments.
- Automate database backups and rehearse restoration procedures.
- Add monitoring and alerting for application health, queue backlogs, and failed background tasks.
- Consider FHIR-based interoperability more concretely if NexClinic is extended toward integration with other healthcare systems [38], [39].
- Add localization support for languages beyond the current interface language.

These recommendations are not commitments and are not currently implemented; they are offered as reasonable directions for a team continuing this project.

XXIII. Glossary

Term	Definition
API	Application Programming Interface. A defined way for one piece of software to request data or actions from another.
ASGI	Asynchronous Server Gateway Interface. A Python server interface standard supporting asynchronous features such as WebSockets, used to serve the Channels layer [18].
Celery	A distributed task-queue library used to run background and scheduled work outside the request/response cycle [21].
CORS	Cross-Origin Resource Sharing. A browser security mechanism controlling which origins may make requests to an API.
CSRF	Cross-Site Request Forgery. An attack pattern, and the corresponding protection mechanism, against unwanted state-changing requests made on a user's behalf.
Django	A Python web framework providing the application structure, ORM, and administration interface used by the NexClinic backend [9].
Django Channels	An extension to Django adding support for WebSockets and other asynchronous protocols over ASGI [18].
Django REST Framework	A toolkit built on Django for constructing REST APIs, including serialization and permission handling [14].

Term	Definition
HTTP-Only Cookie	A cookie flag preventing client-side JavaScript from reading the cookie's value, reducing exposure to cross-site scripting.
JWT	JSON Web Token. A compact, signed token format used to represent an authenticated session [25].
Migration	A versioned, incremental change to the database schema managed by Django's migration framework.
Next.js	A React framework providing routing, server-side rendering, and API route handlers for the NexClinic frontend [3].
OTP	One-Time Password. A short, time-limited numeric code used to verify identity during registration or login.
PostgreSQL	An open-source relational database system, the primary data store targeted by NexClinic [23].
Redis	An in-memory data store used as a message broker for Celery and as a channel-layer backend for Channels [22].
REST	Representational State Transfer. An architectural style for web APIs organized around resources and standard HTTP methods.
Role-Based Access Control	A security approach in which permissions are determined by a user's assigned role.
Serializer	A Django REST Framework component that converts between Python model instances and JSON representations, and validates incoming data.
SMTP	Simple Mail Transfer Protocol. The protocol used to send outbound email, including OTP and notification messages.
WebSocket	A protocol providing a persistent, bidirectional connection between browser and server, used for chat and live notifications [20].
WSGI	Web Server Gateway Interface. The traditional, synchronous Python web server interface, unable to serve Channels WebSocket connections on its own.
WSS	The encrypted, production-appropriate form of the WebSocket protocol, analogous to HTTPS for HTTP.

Table 13. Glossary of terms used in this guide

XXIV. References

The following references support the technical statements made throughout this guide. Every source listed here is cited at least once in the preceding sections, and every numbered citation in the text corresponds to an entry below.

[1] IEEE, "IEEE Editorial Style Manual," IEEE Author Center. [Online]. Available: <https://journals.ieeeauthorcenter.ieee.org/create-your-ieee-journal-article/authoring-tools-and-templates/ieee-editorial-style-manual/>. [Accessed: 10 September 2026].

- [2] IEEE, "IEEE Author Center." [Online]. Available: <https://journals.ieeeauthorcenter.ieee.org/>. [Accessed: 10 September 2026].
- [3] Vercel, "Next.js documentation." [Online]. Available: <https://nextjs.org/docs>. [Accessed: 12 August 2026].
- [4] Vercel, "App Router," Next.js documentation. [Online]. Available: <https://nextjs.org/docs/app>. [Accessed: 12 August 2026].
- [5] Vercel, "Authentication," Next.js documentation. [Online]. Available: <https://nextjs.org/docs/app/building-your-application/authentication>. [Accessed: 15 August 2026].
- [6] Meta Platforms, Inc., "React documentation." [Online]. Available: <https://react.dev/>. [Accessed: 10 August 2026].
- [7] Microsoft, "TypeScript documentation." [Online]. Available: <https://www.typescriptlang.org/docs/>. [Accessed: 11 August 2026].
- [8] Tailwind Labs, "Tailwind CSS documentation." [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: 13 August 2026].
- [9] Django Software Foundation, "Django documentation." [Online]. Available: <https://docs.djangoproject.com/>. [Accessed: 5 August 2026].
- [10] Django Software Foundation, "Security in Django," Django documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/topics/security/>. [Accessed: 6 August 2026].
- [11] Django Software Foundation, "User authentication in Django," Django documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/topics/auth/>. [Accessed: 7 August 2026].
- [12] Django Software Foundation, "Managing files," Django documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/topics/http/file-uploads/>. [Accessed: 8 August 2026].
- [13] Django Software Foundation, "Deployment checklist," Django documentation. [Online]. Available: <https://docs.djangoproject.com/en/stable/howto/deployment/checklist/>. [Accessed: 9 August 2026].
- [14] Encode OSS Ltd., "Django REST Framework documentation." [Online]. Available: <https://www.django-rest-framework.org/>. [Accessed: 16 August 2026].
- [15] Encode OSS Ltd., "Authentication," Django REST Framework documentation. [Online]. Available: <https://www.django-rest-framework.org/api-guide/authentication/>. [Accessed: 17 August 2026].
- [16] Encode OSS Ltd., "Permissions," Django REST Framework documentation. [Online]. Available: <https://www.django-rest-framework.org/api-guide/permissions/>. [Accessed: 18 August 2026].
- [17] Simple JWT maintainers, "Django REST Framework Simple JWT documentation." [Online]. Available: <https://django-rest-framework-simplejwt.readthedocs.io/>. [Accessed: 19 August 2026].
- [18] Django Channels maintainers, "Django Channels documentation." [Online]. Available: <https://channels.readthedocs.io/>. [Accessed: 20 August 2026].

- [19] Django Channels maintainers, "Authentication," Django Channels documentation. [Online]. Available: <https://channels.readthedocs.io/en/stable/topics/authentication.html>. [Accessed: 21 August 2026].
- [20] IETF, "The WebSocket Protocol," RFC 6455, Dec. 2011. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6455>. [Accessed: 22 August 2026].
- [21] Celery Project, "Celery documentation." [Online]. Available: <https://docs.celeryq.dev/>. [Accessed: 23 August 2026].
- [22] Redis Ltd., "Redis documentation." [Online]. Available: <https://redis.io/docs/>. [Accessed: 24 August 2026].
- [23] PostgreSQL Global Development Group, "PostgreSQL documentation." [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: 25 August 2026].
- [24] IETF, "HTTP Semantics," RFC 9110, Jun. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9110>. [Accessed: 26 August 2026].
- [25] IETF, "JSON Web Token (JWT)," RFC 7519, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>. [Accessed: 27 August 2026].
- [26] IETF, "Best Current Practice for OAuth 2.0 Security," RFC 9700, Jan. 2025. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9700>. [Accessed: 28 August 2026].
- [27] OWASP Foundation, "OWASP Application Security Verification Standard." [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>. [Accessed: 29 August 2026].
- [28] OWASP Foundation, "Authentication Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. [Accessed: 30 August 2026].
- [29] OWASP Foundation, "Authorization Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html. [Accessed: 31 August 2026].
- [30] OWASP Foundation, "Password Storage Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. [Accessed: 2 September 2026].
- [31] OWASP Foundation, "Forgot Password Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html. [Accessed: 3 September 2026].
- [32] OWASP Foundation, "Session Management Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. [Accessed: 4 September 2026].
- [33] OWASP Foundation, "File Upload Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html. [Accessed: 5 September 2026].

[34] OWASP Foundation, "WebSocket Security Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html. [Accessed: 6 September 2026].

[35] OWASP Foundation, "Logging Cheat Sheet," OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html. [Accessed: 7 September 2026].

[36] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG)." [Online]. Available: <https://www.w3.org/WAI/standards-guidelines/wcag/>. [Accessed: 8 September 2026].

[37] World Wide Web Consortium, "WAI Forms Tutorial." [Online]. Available: <https://www.w3.org/WAI/tutorials/forms/>. [Accessed: 9 September 2026].

[38] HL7 International, "FHIR Overview." [Online]. Available: <https://www.hl7.org/fhir/overview.html>. [Accessed: 15 September 2026].

[39] World Health Organization, "Digital health." [Online]. Available: <https://www.who.int/health-topics/digital-health>. [Accessed: 18 September 2026].

Screenshot Reference

The following screenshot spaces cover the interfaces and diagnostic outputs referenced throughout this guide.

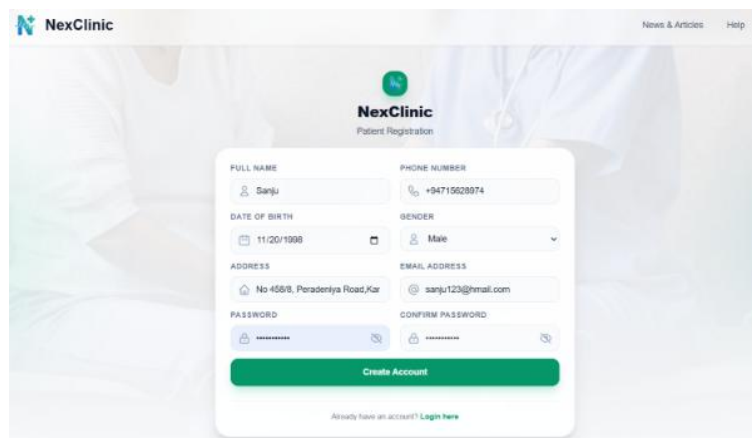
A screenshot of the NexClinic Patient Registration form. The form is titled "NexClinic Patient Registration" and features a green "Create Account" button. The form fields include: Full Name (Sanju), Phone Number (+94715628074), Date of Birth (11/20/1998), Gender (Male), Address (No 45B/8, Peradeniya Road, Kar), Email Address (sanju123@gmail.com), Password, and Confirm Password. A "Login here" link is provided at the bottom for users who already have an account.

Fig. 10. Registration interface of patients

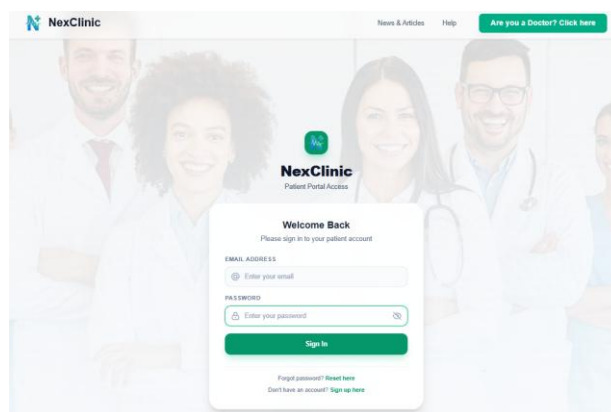
A screenshot of the NexClinic Patient Portal Access login form. The form is titled "NexClinic Patient Portal Access" and features a green "Sign In" button. The form fields include: Email Address (Enter your email) and Password (Enter your password). A "Sign In" link is provided at the bottom for users who already have an account. A "Forgot password? Reset here" link is also present.

Fig. 11.1. Login interface for patients

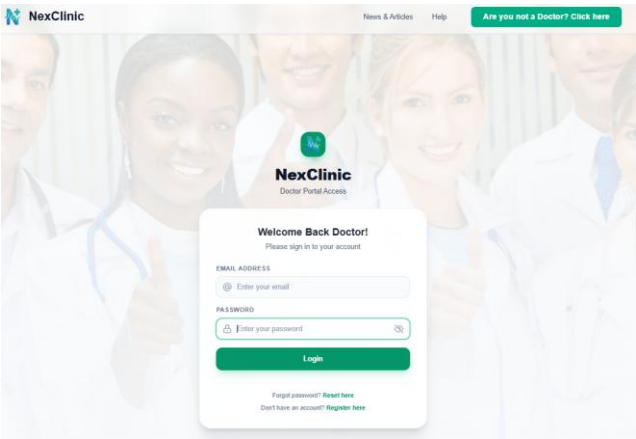


Fig. 11.2. Login interface for doctors

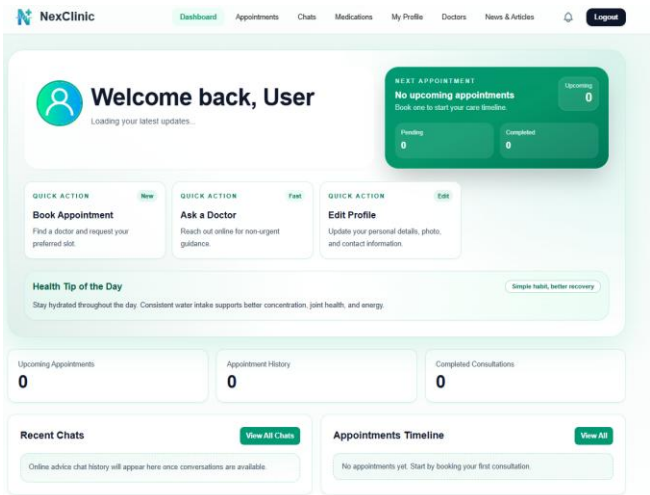


Fig. 12. Patient dashboard

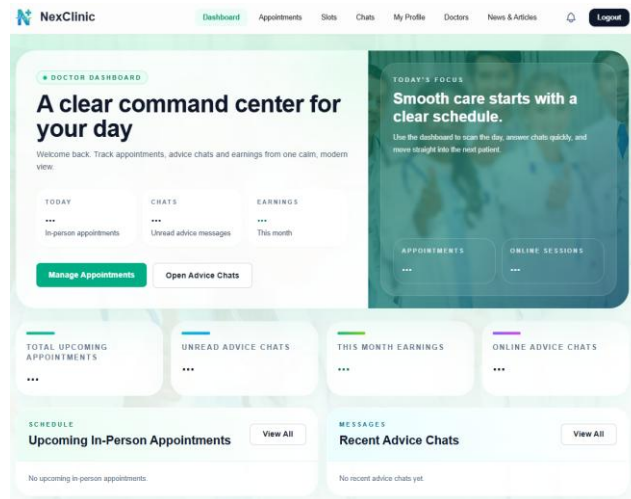


Fig. 13. Doctor dashboard

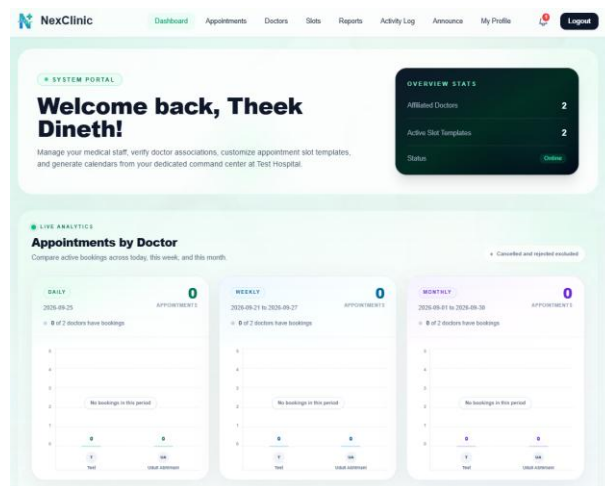


Fig. 14. Hospital administrator dashboard

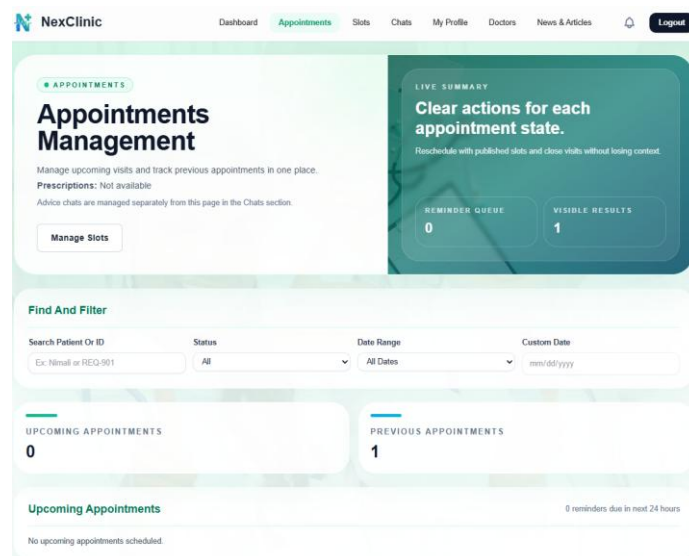


Fig. 15. Doctor appointment workspace

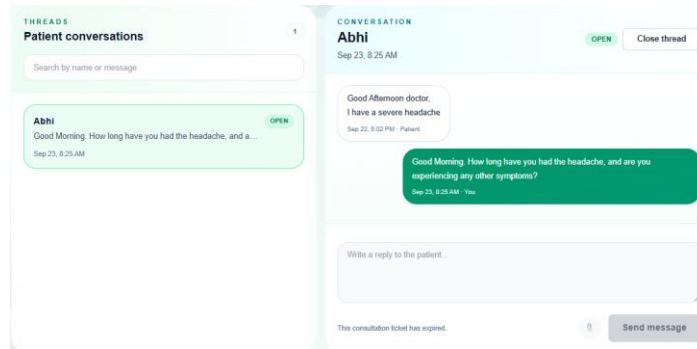


Fig. 16 Advice chat workspace

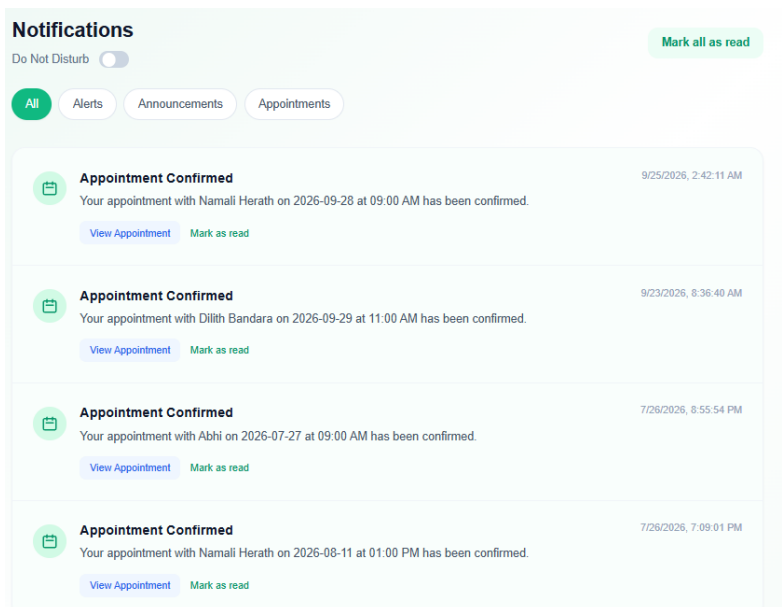


Fig. 17. Notification centre

▼ General	
Request URL	http://localhost:3000/api/patient/appointments
Request Method	GET
Status Code	● 200 OK
Remote Address	[::1]:3000
Referrer Policy	strict-origin-when-cross-origin

Fig. 18. API request example

▼ Response headers	☐ Raw
Connection	keep-alive
Content-Type	application/json
Date	Fri, 25 Sep 2026 04:07:48 GMT
Keep-Alive	timeout=5
Transfer-Encoding	chunked
Vary	rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch

F

Fig. 19. API response example

```
(.venv) PS D:\github_repos\e23-co2060-NexClinic\backend> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, chat, contenttypes, doctor, hospital, notifications, patient, sessions, users
Running migrations:
  No migrations to apply.
```

Fig. 20. Database migration output

```
(.venv) PS D:\github_repos\e23-co2060-NexClinic\backend> python manage.py test doctor.tests.DoctorProfileViewTests.test_doctor_profile_update_rejects_duplicate_email -v 2
Found 1 test(s).
Using existing test database for alias 'default' ('test_postgres')...
Operations to perform:
  Synchronize unmigrated apps: channels, corsheaders, daphne, messages, rest_framework, staticfiles
  Apply all migrations: admin, auth, chat, contenttypes, doctor, hospital, notifications, patient, sessions, users
Synchronizing apps without migrations:
  Creating tables...
  Running deferred SQL...
Running migrations:
  No migrations to apply.
System check identified no issues (0 silenced).
test_doctor_profile_update_rejects_duplicate_email (doctor.tests.DoctorProfileViewTests.test_doctor_profile_update_rejects_duplicate_email) ... ok
-----
Ran 1 test in 5.211s

OK
Preserving test database for alias 'default' ('test_postgres')...
```

Fig. 21. Backend test output

```
(.venv) PS D:\github_repos\e23-co2060-NexClinic\frontend> cd D:\github_repos\e23-co2060-NexClinic\frontend
>> npm run lint
> client@0.1.0 lint
> eslint
```

Fig. 22. Frontend lint output

```
(.venv) PS D:\github_repos\e23-co2060-NexClinic\frontend> cd D:\github_repos\e23-co2060-NexClinic\frontend
>> npm run build

> client@0.1.0 build
> next build

▲ Next.js 16.2.10 (Turbopack)
- Environments: .env

Creating an optimized production build ...
✓ Compiled successfully in 20.8s
✓ Finished TypeScript in 8.7s
✓ Collecting page data using 15 workers in 1468ms
✓ Generating static pages using 15 workers (92/92) in 621ms
✓ Finalizing page optimization in 26ms

Route (app)
├─ /
├─ /_not-found
├─ /admin
├─ /admin/doctors
├─ /admin/settings
├─ /admin/users
├─ /api/auth
├─ /api/auth/doctor/login
├─ /api/auth/doctor/register
├─ /api/auth/hospital/login
├─ /api/auth/hospital/register
├─ /api/auth/login
├─ /api/auth/login/resend-otp
├─ /api/auth/login/verify-otp
├─ /api/auth/logout
├─ /api/auth/register
├─ /api/auth/resend-otp
├─ /api/auth/reset-password/confirm
├─ /api/auth/reset-password/request
├─ /api/auth/set-tokens
├─ /api/auth/verify-otp
├─ /api/chat/slots/doctor/[doctorId]
├─ /api/chat/threads
├─ /api/chat/threads/[threadId]/messages
├─ /api/chat/threads/[threadId]/messages/upload
├─ /api/chat/threads/[threadId]/status
├─ /api/doctor/appointment-slots
├─ /api/doctor/appointment-slots/[slotId]
├─ /api/doctor/appointments
├─ /api/doctor/appointments/[appointmentId]/action
├─ /api/doctor/appointments/[appointmentId]/medical-record
```

Fig. 23. Frontend build output

↔ 12/?token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...Q2I...	101	websocket	D:\github_repos\e23-co2060-Ne	0.0 kB	Pending
↔ notifications/?token=eyJhbGciOiJIUzI1NiIsInR5cCI6I...Q2InO...	101	websocket	D:\github_repos\e23-co2060-Ne	0.0 kB	Pending

Fig. 24. WebSocket connection verification

```
(.venv) PS D:\github_repos\e23-co2060-NexClinic\backend> curl.exe -I "http://localhost:3000"
HTTP/1.1 200 OK
Vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch, Accept-Encoding
Link: </images/hexagons.png>; rel=preload; as="image", </images/logo-main.png>; rel=preload; as="image", </images/HealthDoc.avif>; rel=preload; as="image", </images/doc.jpg>; rel=preload; as="image"
Cache-Control: no-cache, must-revalidate
X-Powered-By: Next.js
Content-Type: text/html; charset=utf-8
Date: Fri, 25 Sep 2026 05:31:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Fig. 25. Deployment health check

```

>> .\venv\Scripts\Activate.ps1
>> python manage.py runserver 127.0.0.1:8000nic\backend>
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 25, 2026 - 11:04:14
Django version 6.0.1, using settings 'main.settings'
Starting ASGI/Daphne version 4.0.0 development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
Unauthorized: /api/users/login/
HTTP POST /api/users/login/ 401 [5.24, 127.0.0.1:62363]
INFO 2026-09-25 11:06:51,042 views patient_login.otp_bypassed email=abhimanipererahmu@gmail.com
HTTP POST /api/users/login/ 200 [4.37, 127.0.0.1:49235]
HTTP GET /api/patient/appointments/ 200 [3.03, 127.0.0.1:60178]
HTTP GET /api/patient/appointments/ 200 [3.12, 127.0.0.1:60179]
HTTP GET /api/patient/profile/ 200 [4.63, 127.0.0.1:49235]
HTTP GET /api/patient/profile/ 200 [4.90, 127.0.0.1:60180]
HTTP GET /api/patient/appointments/ 200 [3.09, 127.0.0.1:57469]
HTTP GET /api/patient/profile/ 200 [4.77, 127.0.0.1:57468]
HTTP GET /api/patient/profile/ 200 [4.64, 127.0.0.1:57513]
HTTP GET /api/patient/appointments/ 200 [2.99, 127.0.0.1:57468]
HTTP GET /api/patient/appointments/ 200 [3.08, 127.0.0.1:57469]
HTTP GET /api/patient/profile/ 200 [4.54, 127.0.0.1:57468]
HTTP GET /api/patient/medications/ 200 [3.26, 127.0.0.1:51885]
HTTP GET /api/patient/medications/ 200 [3.40, 127.0.0.1:51886]
HTTP GET /api/patient/profile/ 200 [5.57, 127.0.0.1:57468]
HTTP GET /media/patient_profiles/images.jpg?v=1790283093706 304 [0.01, 127.0.0.1:64940]
HTTP GET /api/patient/profile/ 200 [5.89, 127.0.0.1:63158]

```

Fig. 26. Application logs